

Rising Beyond Borders

Document 22 — Donations & Razorpay Payment Integration

Master Specification + Claude Code Implementation Prompt

Purpose: build the complete donation/payment architecture using Razorpay as the payment provider, while keeping the public donation flow disabled until the final RBB configuration, credentials, legal/content and production-host prerequisites are supplied.

Implementation principle: build the full production-capable code path now, but use placeholders/server configuration for secrets. Never invent or embed Razorpay keys. The existing Resend integration from Document 21 should be reused for donation notifications once its server-side key is configured.

1. Scope

- Donation page and donation form/payment UX.
- One-time donations.
- Recurring donations only if Razorpay/account configuration supports the approved RBB flow and RBB explicitly enables them.
- Donation amount selection and custom amount.
- Donor details.
- Server-side Razorpay order creation.
- Razorpay Checkout integration.
- Server-side payment signature verification.
- Webhook verification and processing.
- Idempotent payment/order state handling.
- Success, failure, cancellation and retry flows.
- Donation confirmation emails through the existing Resend architecture.
- Admin-independent payment status architecture.
- Security, CSP, environment and secret handling.
- Test/live separation.
- Reconciliation-ready transaction records/logs without introducing a database unless required.

2. Architecture

Razorpay credentials and payment verification must remain server-side. The browser may load Razorpay Checkout using the provider's public checkout mechanism, but it must never contain the Razorpay secret key.

- Browser → donation configuration → server creates Razorpay order → browser opens Razorpay Checkout → Razorpay returns checkout result → server verifies signature → final donation state.
- Razorpay webhook → server verifies webhook signature → idempotently updates transaction state → optional Resend notification.
- The client success screen is never treated as proof of payment.
- Webhook/server verification is authoritative.
- The server should return only safe payment status/reference information to the browser.

3. Environment Variables

Variable	Visibility	Purpose
RAZORPAY_KEY_ID	Public/server config	Razorpay Checkout key ID; may be exposed to browser when required by Checkout.
RAZORPAY_KEY_SECRET	Server secret	Create orders and server-side verification.
RAZORPAY_WEBHOOK_SECRET	Server secret	Verify Razorpay webhook signatures.
RAZORPAY_MODE	Server config	test/live; fail closed if invalid.
DONATION_ENABLED	Server/public config	Controls whether production donation action is available.
DONATION_CURRENCY	Server/public config	Approved currency, defaulting to INR only when RBB approves.
DONATION_MIN_AMOUNT	Server/public config	Approved minimum donation amount.
DONATION_MAX_AMOUNT	Server/public config	Approved maximum donation amount.
DONATION_ALLOWED_AMOUNTS	Server/public config	Optional approved preset amounts.
DONATION_EMAIL_ENABLED	Server config	Controls Resend donation notifications.
EMAIL_API_KEY	Server secret	Existing Resend server-side key from Document 21.
EMAIL_FROM	Server config	Approved RBB sending address.
EMAIL_TO_DONATIONS	Server config	Approved RBB donation notification inbox.

Never commit secret values. Never place RAZORPAY_KEY_SECRET, RAZORPAY_WEBHOOK_SECRET or EMAIL_API_KEY in VITE_* variables or client bundles.

4. Donation State Model

State	Meaning	Customer-facing behavior
created	Donation intent/order created.	Checkout can be opened.
checkout_opened	Checkout initiated.	Payment UI active.
authorized	Provider indicates authorization; not necessarily final settlement.	Show processing/verification state.
captured	Verified successful payment.	Success/receipt flow.
failed	Payment failed.	Explain failure and offer retry.
cancelled	Donor exited/cancelled.	Offer retry without claiming failure.
refunded	Verified refund recorded.	Show refund status where applicable.
unknown	State cannot safely be determined.	Never claim success; direct to support path if approved.

5. Amount & Currency Rules

- Do not hard-code final donation amounts until RBB approves them.
- Do not display a currency symbol merely because the frontend needs a placeholder.
- The server is authoritative for currency and allowed amount constraints.
- Convert the approved human amount into the provider's smallest currency unit exactly once.
- Never trust a client-supplied amount, currency or order ID.
- Reject amounts below/above server-side limits.
- Reject unsupported currency values.
- If custom amounts are enabled, enforce numeric precision appropriate to the currency.
- Never use floating-point arithmetic for monetary calculations; use integer minor units/decimal-safe logic.

6. Donor Information

- Collect only information needed for the approved donation process and receipt requirements.
- Suggested fields: name, email, optional phone, amount, currency and optional donation designation.
- Do not add address, PAN, tax identifiers or other sensitive fields unless RBB explicitly requires them and the legal/receipt design supports them.
- Do not put donor personal data into URLs.
- Do not put full donor information into logs.
- Do not expose donor information through payment-status APIs to arbitrary clients.

7. Razorpay Order Creation

- Create orders only server-side.
- Generate a unique internal donation reference.
- Generate/accept a safe idempotency key for repeated client requests.
- The server calculates the final amount and currency.
- Attach a non-sensitive internal donation reference in provider order metadata/notes where appropriate.
- Never trust amount/currency values returned from the browser.
- Return only the order ID, public key ID, amount, currency and safe reference required by Checkout.

8. Payment Verification

The browser callback is useful for UX but is not authoritative. The server must verify the Razorpay payment signature using the server-side secret and the expected order/payment relationship before marking a donation successful.

- Verify the signature server-side.
- Verify the payment belongs to the expected order.
- Verify the expected amount and currency where provider data permits.
- Do not mark captured merely because Checkout returned a success callback.
- Use provider/webhook confirmation for asynchronous state changes.
- Make verification idempotent.

9. Webhook Architecture

- Create a dedicated server endpoint for Razorpay webhooks.
- Verify the webhook signature using RAZORPAY_WEBHOOK_SECRET before parsing it as trusted state.
- Process supported payment events explicitly; reject/ignore unknown events safely.
- Use idempotency so duplicate webhook delivery cannot create duplicate donation records/emails.
- Store enough non-sensitive transaction metadata to reconcile provider events with internal references.
- Never mark a donation successful based on an unverified webhook.
- Do not expose webhook payloads through logs or browser responses.

10. Email Integration with Existing Resend Pipeline

- Reuse the Document 21 Resend infrastructure; do not create a second email client.
- Create donation-specific templates: donation received/success, donation processing if needed, donation failed/cancelled where appropriate, and internal donation notification.
- Only send a donor success/receipt email after authoritative payment verification.
- Do not send a 'successful donation' email from the browser callback alone.
- Avoid including unnecessary payment metadata.
- Never email secret keys, signatures or full provider payloads.
- Use the same safe HTML + plain-text template approach established in Document 21.

11. Donation Email Templates

Template	Recipient	Content
Donation success	Donor	Thank you → verified amount/currency → donation reference → approved purpose wording → receipt/t
Donation notification	RBB	Verified donation → donor details approved for internal notification → amount/currency → payment refe
Donation processing	Donor	Payment received/verification pending → reference → no claim of successful completion.
Donation failed	Donor	Payment was not completed → safe retry wording → support path only if approved.
Donation refunded	Donor	Verified refund → amount/reference → approved refund wording.

12. Refunds & Reconciliation

- A refund must never be represented as successful merely because an internal operator clicked a UI action.
- Use Razorpay's authoritative refund state/webhook information.
- Keep original donation reference linked to refund reference.
- Design transaction handling so duplicate refund events are harmless.
- The implementation must expose enough information for later reconciliation without storing unnecessary personal/payment data.
- Do not build an admin dashboard in this phase.

13. Recurring Donations

Build the architecture so recurring donations can be added without redesign, but do not enable recurring payments unless RBB explicitly approves them and the Razorpay account/payment method supports the required recurring workflow.

- Do not fake recurring billing by scheduling repeated one-time charges.
- Do not store card data or bank credentials.
- Treat provider mandates/subscriptions as provider-controlled state.
- Recurring-specific webhook events must be verified and idempotent.
- Recurring donor communications need separate approved wording.

14. Donation UX

- Donation page clearly explains what the visitor is doing before checkout.
- Amount selection is accessible by keyboard and screen reader.
- Custom amount input has clear validation.
- Payment launch state prevents accidental duplicate order creation.
- Success page distinguishes 'payment verified' from 'processing'.
- Cancel returns the donor to a safe retry state.
- Failure provides a retry action without implying funds were charged when not verified.
- No sensitive payment information is displayed or collected by the RBB frontend beyond provider-required fields.
- Do not display unsupported tax-deductibility/receipt claims.

15. CSP & Third-Party Origins

- Razorpay Checkout's actual required browser origin(s) must be verified against the current Razorpay integration documentation during implementation.
- Do not guess provider origins or add broad CSP wildcards.
- Add only the exact script/connect/frame/font/image origins actually required by Razorpay Checkout.
- Update the build-time approved-origin enforcement and technology audit.
- The payment server endpoints remain same-origin where possible.
- Resend remains server-to-server and does not need browser CSP access.

16. Test/Live Separation

- Default donation mode is OFF.
- Test mode uses Razorpay test credentials and test endpoints/Checkout configuration.
- Live mode requires explicit production configuration.
- A production build with missing live credentials must fail closed or keep donation disabled.
- Never fall back from live to test silently.
- Email mode follows Document 21 and must not send real donor emails during local tests.
- Use test webhook secrets and endpoints separately from production.

17. Security Requirements

- Razorpay secret key never reaches browser.
- Webhook secret never reaches browser.
- Amounts are calculated server-side.
- Currency is validated server-side.
- Order IDs are verified server-side.
- Payment signatures are verified server-side.
- Webhook signatures are verified server-side.
- Duplicate requests/events are idempotent.
- No payment data is stored unnecessarily.
- No card/bank credentials are stored by RBB.
- No secrets are logged.
- Provider errors are not returned verbatim to users.
- Donation APIs are rate-limited.
- Donation endpoints reject invalid origins/content types/methods.
- CSP is updated only for exact provider origins.

18. Suggested Server Endpoints

- `POST /api/donations/order` — validate donation intent and create provider order.
- `POST /api/donations/verify` — verify Checkout signature and payment/order relationship.
- `POST /api/donations/webhook` — verify and process provider events.
- `GET /api/donations/status/:reference` — only if needed; return minimal safe status and require an unguessable reference/token.
- Do not expose arbitrary transaction lookup by Razorpay payment ID.

19. Claude Code Implementation Prompt

You are implementing Document 22 – Donations & Razorpay Payment Integration for the Rising Beyond Borders website.

GOAL

Build the full donation/payment code path using Razorpay, while keeping production donation activation disabled until RBB supplies/approves the final configuration and credentials.

PROVIDER

Razorpay is the selected payment provider for this phase.

IMPORTANT

- Do not ask for or invent Razorpay keys.
- Build the code using environment variable names/placeholders.
- Never commit secret values.
- Do not expose `RAZORPAY_KEY_SECRET` or `RAZORPAY_WEBHOOK_SECRET` to the browser.
- Reuse the existing Document 21 Resend server-side email infrastructure.
- Do not duplicate the Resend client.
- Do not stage or commit.

ENVIRONMENT

Server-side:

RAZORPAY_KEY_ID

RAZORPAY_KEY_SECRET

RAZORPAY_WEBHOOK_SECRET

RAZORPAY_MODE

DONATION_ENABLED

DONATION_CURRENCY

DONATION_MIN_AMOUNT

DONATION_MAX_AMOUNT

DONATION_ALLOWED_AMOUNTS

DONATION_EMAIL_ENABLED

EMAIL_API_KEY

EMAIL_FROM

EMAIL_TO_DONATIONS

CLIENT may receive only the public Razorpay key ID and safe donation configuration necessary for Checkout.

ARCHITECTURE

Browser -> POST /api/donations/order -> server -> Razorpay order

Browser -> Razorpay Checkout

Browser callback -> POST /api/donations/verify -> server verification

Razorpay webhook -> POST /api/donations/webhook -> signature verification -> authoritative state

Verified success -> Resend donation email

AUTHORITATIVE RULE

Never mark a donation successful based only on the client callback. Server-side signature verification and verified provider/webhook state are authoritative.

AMOUNTS

- Server calculates/validates amount.
- Use integer minor units.
- Never trust browser amount/currency.
- Reject unsupported currency and invalid bounds.
- Keep final amount/currency configurable.

PAYMENT STATES

created, checkout_opened, authorized, captured, failed, cancelled, refunded, unknown.

IDEMPOTENCY

Implement idempotency for order creation, verification and webhook processing. Duplicate requests/events must not produce duplicate successful donation emails.

WEBHOOK

Verify Razorpay webhook signature before trusting payload. Handle supported events explicitly. Ignore/reject unknown events safely.

EMAIL

Reuse existing Resend infrastructure.

Create:

- donation success email
- internal donation notification
- processing email if useful
- failure/cancel email if approved
- refund email

Use branded HTML + plain text.

Never send success email until authoritative payment verification.

CSP

Inspect the actual Razorpay Checkout network requirements and add only exact required origins to the CSP. Do not use wildcards. Update technology audit.

UX

- Accessible amount selector
- Custom amount validation
- Loading/prevent duplicate order creation
- Checkout launch
- Processing state
- Success state only after verification
- Cancel/retry
- Failure/retry
- No unsupported tax/receipt claims

TEST/LIVE

Default OFF.

Test mode uses Razorpay test credentials.

Live mode requires explicit production configuration.

Missing credentials must fail closed.

Never silently fall back between test and live.

SECURITY

- Secret scan
- No secrets in client bundle
- Server validation
- Rate limiting
- Signature verification
- Webhook verification
- Idempotency
- Generic errors
- No sensitive logs
- CSP regression
- Content/SEO/accessibility regression

DO NOT

- Add admin dashboard.
- Add a database unless a concrete provider requirement makes it necessary and report it first.
- Store card/bank details.
- Invent donation amounts.
- Invent currency.
- Invent tax benefits.
- Invent refund policy.
- Invent receipt wording.
- Invent RBB payment recipient details.

REPORT

Return:

- files changed
- endpoints created
- Razorpay integration status
- environment variable names
- donation states
- idempotency approach
- webhook events handled

- email templates
- CSP origins added
- tests and results
- test/live separation
- unresolved RBB decisions
- whether secrets appear anywhere
- whether anything was staged or committed.

20. RBB Decisions Required Before Live Donations

- Final production Razorpay account.
- Production Razorpay Key ID/Secret/Webhook Secret.
- Donation currency.
- Preset donation amounts.
- Minimum/maximum amount.
- Custom amount enabled or disabled.
- One-time donations enabled.
- Recurring donations enabled/disabled.
- Donation purpose/designation options.
- Donor fields.
- Tax/receipt wording, if applicable.
- Refund wording/policy.
- Approved donation From address.
- Donation notification recipient.
- Donor acknowledgement wording.
- Privacy Policy publication.
- Production host/serverless runtime.
- Production rate limiting/shared store.

21. Acceptance Criteria

- Razorpay order creation is server-side.
- Razorpay secrets never reach the client.
- Payment signatures are verified server-side.
- Webhook signatures are verified server-side.
- Payment state is idempotent.
- Client callback cannot falsely mark payment successful.
- Donation success email is sent only after authoritative verification.
- All donation emails use structured templates.
- Test/live modes are separated and fail closed.
- No secrets appear in repository or client build.
- CSP contains only verified Razorpay origins.

- Accessibility and existing site regression tests remain clean.
- No unsupported tax, receipt, refund or organizational claims are introduced.
- Donation remains disabled until final RBB activation decisions are satisfied.

Document status: Implementation specification. Razorpay is selected as the provider. Production activation remains gated by RBB configuration, credentials, legal/content decisions and production-host readiness.